

TỔNG HỢP MÃ NGUỒN CÁC GIẢI THUẬT ĐỒ THỊ

Tài liệu đính kèm chứa mã nguồn Python chuẩn hóa của 8 giải thuật cốt lõi

1. Giải thuật duyệt đồ thị: BFS và DFS

Mã nguồn cài đặt hai giải thuật duyệt đồ thị cơ bản: Breadth-First Search (BFS) sử dụng Queue tuần tự và Depth-First Search (DFS) sử dụng đệ quy hệ thống. Các đỉnh lân cận được sắp xếp theo thứ tự nhãn tăng dần để đảm bảo kết quả chạy vết đồng nhất.

```
from collections import deque

def bfs_generator(graph, start):
    """
    Generator version of BFS for GUI animation.
    Yields ("START"|"STEP"|"FINISHED", data)
    """
    if start not in graph:
        return None, "Đỉnh bắt đầu không có trong đồ thị."
    visited = []
    visited_set = {start}
    queue = deque([start])
    edges = []

    yield "START", (list(visited), list(edges), list(queue))

    while queue:
        u = queue.popleft()
        visited.append(u)
        yield "STEP", (list(visited), list(edges), list(queue), u, None)

        # Sort neighbors by vertex index for a deterministic traversal order
        for v, w in sorted(graph.get(u, []), key=lambda x: x[0]):
            if v not in visited_set:
                visited_set.add(v)
                queue.append(v)
                edges.append((u, v, w))
                yield "STEP", (list(visited), list(edges), list(queue), u, (u, v, w))

    yield "FINISHED", (visited, edges)

def dfs_generator(graph, start):
    """
    Generator version of DFS for GUI animation.
    Yields ("START"|"STEP"|"FINISHED", data)
    """
    if start not in graph:
        return None, "Đỉnh bắt đầu không có trong đồ thị."
    visited = []
    visited_set = {start}
    edges = []

    neighbors = {u: sorted(graph.get(u, []), key=lambda x: x[0]) for u in graph}
    stack = [[start, 0]]
    visited.append(start)

    yield "START", (list(visited), list(edges), [s[0] for s in stack])

    while stack:
        u, idx = stack[-1]
        u_neighbors = neighbors.get(u, [])
        if idx < len(u_neighbors):
            v, w = u_neighbors[idx]
            stack[-1][1] += 1
            if v not in visited_set:
                visited_set.add(v)
                visited.append(v)
                edges.append((u, v, w))
                yield "STEP", (list(visited), list(edges), [s[0] for s in stack], u, (u, v, w))
                stack.append([v, 0])
```

```

        else:
            stack.pop()
            yield "STEP", (list(visited), list(edges), [s[0] for s in stack], u, None)

    yield "FINISHED", (visited, edges)

def bfs(graph, start):
    """
    Wrapper version of BFS returning (visited_nodes, traversal_edges).
    """
    gen = bfs_generator(graph, start)
    last_val = ([], [])
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val

def dfs(graph, start):
    """
    Wrapper version of DFS returning (visited_nodes, traversal_edges).
    """
    gen = dfs_generator(graph, start)
    last_val = ([], [])
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val

```

2. Kiểm tra đồ thị hai phía (Bipartite Graph Check)

Mã nguồn kiểm tra đồ thị hai phía sử dụng thuật toán tô hai màu (2-Coloring) thông qua duyệt DFS. Nếu phát hiện chu trình độ dài lẻ (odd cycle), giải thuật sẽ truy vết và trả về chu trình đó làm minh chứng mâu thuẫn.

```
def check_bipartite_generator(graph):
    """
    Generator version of Bipartite graph check for GUI animation.
    Yields ("START"|"COLOR"|"CONFLICT"|"FINISHED", data)
    """
    color = {}
    parent = {}

    yield "START", (dict(color), None)

    for start_node in graph:
        if start_node not in color:
            # Iterative DFS for coloring and odd cycle detection
            stack = [(start_node, 0)]
            color[start_node] = 0
            yield "COLOR", (dict(color), start_node, 0, None)

            while stack:
                u, c = stack[-1]
                unvisited_found = False
                for v, _ in graph[u]:
                    if v not in color:
                        color[v] = 1 - c
                        parent[v] = u
                        stack.append((v, 1 - c))
                        unvisited_found = True
                        yield "COLOR", (dict(color), v, 1 - c, (u, v))
                        break
                    elif color[v] == color[u]:
                        # Found an odd cycle (conflict)
                        cycle = [v, u]
                        curr = u
                        while curr in parent and curr != v:
                            curr = parent[curr]
                            cycle.append(curr)
                        cycle = cycle[::-1]
                        yield "CONFLICT", (dict(color), cycle, (u, v))
                        yield "FINISHED", (False, cycle)
                        return

                if not unvisited_found:
                    stack.pop()

    yield "FINISHED", (True, color)

def check_bipartite(graph):
    """
    Wrapper version of check_bipartite returning (is_bipartite, color_dict_or_odd_cycle).
    """
    gen = check_bipartite_generator(graph)
    last_val = (True, {})
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
    return last_val
```

3. Tìm đường đi ngắn nhất: Dijkstra

Mã nguồn thuật toán Dijkstra tìm đường đi ngắn nhất từ một đỉnh nguồn tới mọi đỉnh khác trên đồ thị có trọng số không âm. Giải thuật sử dụng cấu trúc hàng đợi ưu tiên (Priority Queue / Heap) để đạt hiệu năng $O(E \log V)$ và kiểm tra lỗi nếu có trọng số âm.

Tên tin nguồn: [algorithms/dijkstra.py](#)

```
import heapq

def dijkstra_generator(graph, start):
    """
    Generator version of Dijkstra's algorithm for GUI animation.
    Yields ("START"|"EXAMINE"|"RELAX"|"SKIP"|"FINISHED", data)
    """
    dist = {node: float("inf") for node in graph}
    prev = {node: None for node in graph}
    dist[start] = 0
    pq = [(0, start)]

    yield "START", (dict(dist), dict(prev), start)

    while pq:
        current_dist, u = heapq.heappop(pq)
        yield "EXAMINE", (dict(dist), dict(prev), u)

        if current_dist > dist[u]:
            continue

        for v, weight in graph[u]:
            if weight < 0:
                raise ValueError("Dijkstra does not support negative edge weights.")
            candidate = current_dist + weight
            if candidate < dist[v]:
                dist[v] = candidate
                prev[v] = u
                heapq.heappush(pq, (candidate, v))
                yield "RELAX", (dict(dist), dict(prev), (u, v, weight, candidate))
            else:
                yield "SKIP", (dict(dist), dict(prev), (u, v, weight))

    yield "FINISHED", (dist, prev)

def dijkstra(graph, start):
    """
    Wrapper version of Dijkstra's algorithm returning (dist, prev).
    """
    gen = dijkstra_generator(graph, start)
    last_val = ({}, {})
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
    return last_val
```

4. Cây khung nhỏ nhất: Kruskal

Mã nguồn thuật toán Kruskal tìm cây khung cực tiểu (MST). Thuật toán tiếp cận theo hướng duyệt cạnh (Edge-based): sắp xếp các cạnh theo trọng số tăng dần và dùng cấu trúc dữ liệu Disjoint Set Union (DSU) để kiểm tra chu trình và hợp nhất tập hợp một cách tối ưu.

Tên tin nguồn: [algorithms/kruskal.py](#)

```
def kruskal_generator(node_count, edges):
    """
    Generator version of Kruskal's algorithm for GUI animation.
    Yields ("START"|"STEP"|"FINISHED", data)
    """
    parent = list(range(node_count))

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    sorted_edges = sorted(edges, key=lambda edge: edge[2])
    mst = []
    total = 0

    yield "START", (list(mst), total, None)

    for u, v, w in sorted_edges:
        root_u, root_v = find(u), find(v)
        if root_u != root_v:
            parent[root_u] = root_v
            mst.append((u, v, w))
            total += w
            # Yield step with edge, showing it was accepted
            yield "STEP", (list(mst), total, (u, v, w), True)
        else:
            # Yield step with edge, showing it was rejected (forms cycle)
            yield "STEP", (list(mst), total, (u, v, w), False)

    yield "FINISHED", (list(mst), total, None)

def kruskal(node_count, edges):
    """
    Wrapper version of Kruskal's algorithm returning (mst, total).
    """
    gen = kruskal_generator(node_count, edges)
    last_val = ([], 0)
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val
```

5. Cây khung nhỏ nhất: Prim

Mã nguồn thuật toán Prim tìm cây khung cực tiểu (MST). Thuật toán tiếp cận theo hướng duyệt đỉnh (Vertex-based): loang dần cây khung từ đỉnh gốc bằng cách sử dụng hàng đợi ưu tiên Heap để chọn cạnh ngắn nhất nối từ cây ra ngoài.


```
import heapq

def prim_generator(node_count, graph, start=0):
    """
    Generator version of Prim's algorithm for GUI animation.
    Yields ("START"|"EXAMINE"|"STEP"|"SKIP"|"FINISHED", data)
    """
    if start not in graph:
        return None, "Đỉnh bắt đầu không có trong đồ thị."

    visited = {start}
    mst = []
    total = 0
    pq = []

    # Yield initial state
    yield "START", (list(visited), list(mst), total)

    # Push all edges from the start node
    for v, w in graph.get(start, []):
        heapq.heappush(pq, (w, start, v))

    while pq and len(visited) < node_count:
        w, u, v = heapq.heappop(pq)
        # Yield that we are examining this edge
        yield "EXAMINE", (list(visited), list(mst), (u, v, w))

        if v not in visited:
            visited.add(v)
            mst.append((u, v, w))
            total += w
            # Yield that we accepted this edge and visited v
            yield "STEP", (list(visited), list(mst), (u, v, w))

            for next_v, next_w in graph.get(v, []):
                if next_v not in visited:
                    heapq.heappush(pq, (next_w, v, next_v))
        else:
            # Yield that we skipped this edge because it would form a cycle
            yield "SKIP", (list(visited), list(mst), (u, v, w))

    yield "FINISHED", (list(visited), list(mst), total)

def prim(node_count, graph, start=0):
    """
    Wrapper version of Prim's algorithm returning (mst, total).
    """
    gen = prim_generator(node_count, graph, start)
    last_val = ([], 0)
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[1], data[2])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val
```

6. Luồng cực đại: Edmonds-Karp

Mã nguồn thuật toán Edmonds-Karp tìm luồng cực đại trên mạng luồng có hướng/vô hướng. Đây là phiên bản tối ưu của Ford-Fulkerson sử dụng duyệt BFS để tìm đường tăng luồng có ít cạnh nhất trên đồ thị dư (Residual Graph), đảm bảo dừng trong thời gian $O(VE^2)$.

```

from collections import deque

def bfs_augmenting_path(graph, capacity, flow, s, t, parent):
    visited = {s}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            residual = capacity[(u, v)] - flow.get((u, v), 0)
            if v not in visited and residual > 0:
                visited.add(v)
                parent[v] = u
                if v == t:
                    return True
                queue.append(v)
    return False

def max_flow_generator(node_count, edges, s, t, is_directed=False):
    """
    Generator version of Edmonds-Karp Max Flow algorithm for GUI animation.
    Yields ("START"|"STEP"|"FINISHED", data)
    """
    graph = {i: set() for i in range(node_count)}
    capacity = {}

    for u, v, w in edges:
        graph[u].add(v)
        capacity[(u, v)] = capacity.get((u, v), 0) + w
        if not is_directed:
            graph[v].add(u)
            capacity[(v, u)] = capacity.get((v, u), 0) + w
        else:
            if (v, u) not in capacity:
                graph[v].add(u)
                capacity[(v, u)] = 0

    flow = {}
    max_flow_val = 0
    augmenting_paths = []

    yield "START", (0, {}, [], None)

    parent = {}
    while bfs_augmenting_path(graph, capacity, flow, s, t, parent):
        path_flow = float("inf")
        curr = t
        path = []
        while curr != s:
            p = parent[curr]
            path.append(curr)
            residual = capacity[(p, curr)] - flow.get((p, curr), 0)
            path_flow = min(path_flow, residual)
            curr = p
        path.append(s)
        path = path[::-1]

        curr = t
        while curr != s:
            p = parent[curr]
            flow[(p, curr)] = flow.get((p, curr), 0) + path_flow

```

```

        flow[(curr, p)] = flow.get((curr, p), 0) - path_flow
        curr = p

    max_flow_val += path_flow
    augmenting_paths.append((path, path_flow))

    # Reconstruct current flow dictionary for edges
    curr_flow = {}
    for u_edge, v_edge, w_edge in edges:
        f_val = flow.get((u_edge, v_edge), 0)
        if not is_directed:
            if f_val < 0:
                curr_flow[(v_edge, u_edge)] = -f_val
            elif f_val > 0:
                curr_flow[(u_edge, v_edge)] = f_val
        else:
            if f_val > 0:
                curr_flow[(u_edge, v_edge)] = f_val

    # Yield each augmenting path and the updated flow
    yield "STEP", (max_flow_val, curr_flow, list(path), path_flow)
    parent = {}

orig_flow = {}
for u, v, w in edges:
    f_val = flow.get((u, v), 0)
    if not is_directed:
        if f_val < 0:
            orig_flow[(v, u)] = -f_val
        elif f_val > 0:
            orig_flow[(u, v)] = f_val
    else:
        if f_val > 0:
            orig_flow[(u, v)] = f_val

yield "FINISHED", (max_flow_val, orig_flow, augmenting_paths)

def max_flow(node_count, edges, s, t, is_directed=False):
    """
    Wrapper version of max_flow returning (max_flow_val, orig_flow, augmenting_paths).
    """
    gen = max_flow_generator(node_count, edges, s, t, is_directed)
    last_val = (0, {}, [])
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1], data[2])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val

```

7. Chu trình Euler: Giải thuật Fleury

Mã nguồn tìm đường đi hoặc chu trình Euler sử dụng giải thuật Fleury cho đồ thị vô hướng. Giải thuật kiểm tra tính liên thông và bậc của các đỉnh trước khi đi tham lam, đảm bảo chỉ đi qua cạnh cầu (bridge) khi không còn cạnh nào khác.

Tên tin nguồn: [algorithms/fleury.py](#)

```
def find_eulerian_path_or_circuit_fleury_generator(graph, is_directed=False):
    """
    Hàm Fleury phiên bản Generator phục vụ cho việc tạo Hoạt họa (Animation) trên GUI.
    Mỗi bước lặp trả về (loại_bước, dữ_liệu)
    """
    if is_directed:
        return None, "Giải thuật Fleury chỉ hỗ trợ đồ thị vô hướng."

    degree = {u: len(graph.get(u, [])) for u in graph}
    start_vertex = next((u for u in degree if degree[u] > 0), None)
    if start_vertex is None:
        return [], "Đồ thị không có cạnh."

    # Kiểm tra tính liên thông
    visited = set()
    stack = [start_vertex]
    while stack:
        u = stack.pop()
        if u not in visited:
            visited.add(u)
            for v, _ in graph.get(u, []):
                if v not in visited:
                    stack.append(v)
    if any(degree[u] > 0 and u not in visited for u in degree):
        return None, "Đồ thị không liên thông."

    # Kiểm tra bậc lẻ
    odd_vertices = [u for u in degree if degree[u] % 2 != 0]
    if len(odd_vertices) == 0:
        start_node = start_vertex
        path_type = "Chu trình Euler (Fleury)"
    elif len(odd_vertices) == 2:
        start_node = odd_vertices[0]
        path_type = "Đường đi Euler (Fleury)"
    else:
        return None, f"Đồ thị có {len(odd_vertices)} đỉnh bậc lẻ (Fleury yêu cầu 0 hoặc 2)."

    # Khởi tạo danh sách cạnh với ID duy nhất
    edge_list = []
    edge_id = 0
    for u in graph:
        for v, _ in graph[u]:
            if u < v:
                edge_list.append((edge_id, u, v))
                edge_id += 1
            elif u == v:
                edge_list.append((edge_id, u, u))
                edge_id += 1

    # Dựng sẵn danh sách kề cố định
    full_adj = {}
    for eid, x, y in edge_list:
        full_adj.setdefault(x, []).append((y, eid))
        if x != y:
            full_adj.setdefault(y, []).append((x, eid))

    def dfs_count(u, active_edge_ids):
        visited_nodes = {u}
        stack = [u]
        while stack:
            curr_node = stack.pop()
            for neighbor, eid in full_adj.get(curr_node, []):
```

```

        if eid in active_edge_ids and neighbor not in visited_nodes:
            visited_nodes.add(neighbor)
            stack.append(neighbor)
    return len(visited_nodes)

def is_bridge(u, v, eid, active_edge_ids):
    count1 = dfs_count(u, active_edge_ids)
    count2 = dfs_count(u, active_edge_ids - {eid})
    return count2 < count1

remaining_ids = set(eid for eid, _, _ in edge_list)
curr = start_node
path = [curr]

# Trả về siêu dữ liệu trước khi chạy vòng lặp chính
yield "START", (path, path_type, None)

while remaining_ids:
    # Lấy trực tiếp từ full_adj, tối ưu O(1) bước tìm kiếm
    candidates = [(neighbor, eid) for neighbor, eid in full_adj.get(curr, []) if eid in
remaining_ids]

    if not candidates:
        break

    if len(candidates) == 1:
        next_node, eid = candidates[0]
    else:
        non_bridges = [(neighbor, eid) for neighbor, eid in candidates if not is_bridge(curr,
neighbor, eid, remaining_ids)]
        if non_bridges:
            next_node, eid = non_bridges[0]
        else:
            next_node, eid = candidates[0]

    remaining_ids.remove(eid)
    path.append(next_node)

    # Bắn trạng thái ra cho giao diện cập nhật (Tô màu cạnh vừa đi, highlight đỉnh hiện tại)
    yield "STEP", (list(path), curr, next_node, eid)
    curr = next_node

yield "FINISHED", (path, path_type, None)

def find_eulerian_path_or_circuit_fleury(graph, is_directed=False):
    """
    Hàm bao tương thích ngược để trả về trực tiếp kết quả chạy cuối cùng.
    """
    gen = find_eulerian_path_or_circuit_fleury_generator(graph, is_directed)
    last_finished = None
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_finished = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
    return last_finished

```

8. Chu trình Euler: Giải thuật Hierholzer

Mã nguồn tìm đường đi hoặc chu trình Euler sử dụng giải thuật Hierholzer. Đây là thuật toán tối ưu có độ phức tạp $O(E)$ bằng cách xây dựng và lồng ghép các chu trình con đơn giản thông qua cơ chế Ngăn xếp (Stack).

Tên tin nguồn: algorithms/hierholzer.py

```
def find_eulerian_path_or_circuit_hierholzer_generator(graph, is_directed=False):
    """
    Generator version of Hierholzer's algorithm for GUI animation.
    Yields ("START"|"PUSH"|"POP"|"FINISHED", data)
    """
    if is_directed:
        return None, "Giải thuật Hierholzer chỉ hỗ trợ đồ thị vô hướng."

    # 1. Calculate degrees
    degree = {u: len(graph.get(u, [])) for u in graph}

    # 2. Check connectivity for vertices with edges
    start_vertex = next((u for u in degree if degree[u] > 0), None)
    if start_vertex is None:
        return [], "Đồ thị không có cạnh."

    visited = set()
    stack = [start_vertex]
    while stack:
        u = stack.pop()
        if u not in visited:
            visited.add(u)
            for v, _ in graph.get(u, []):
                if v not in visited:
                    stack.append(v)

    if any(degree[u] > 0 and u not in visited for u in degree):
        return None, "Đồ thị không liên thông (xét các đỉnh có cạnh)."
```

```
    # 3. Find odd degree vertices
    odd_vertices = [u for u in degree if degree[u] % 2 != 0]

    if len(odd_vertices) == 0:
        start_node = start_vertex
        path_type = "Chu trình Euler (Hierholzer)"
    elif len(odd_vertices) == 2:
        start_node = odd_vertices[0]
        path_type = "Đường đi Euler (Hierholzer)"
    else:
        return None, f"Đồ thị có {len(odd_vertices)} đỉnh bậc lẻ (yêu cầu phải là 0 hoặc 2)."
```

```
    # 4. Hierholzer's Algorithm
    adj = {u: {} for u in graph}
    for u in graph:
        for v, w in graph[u]:
            adj[u][v] = adj[u].get(v, 0) + 1

    stack = [start_node]
    path = []

    yield "START", (list(stack), list(path), path_type)

    while stack:
        u = stack[-1]
        neighbors = [v for v in adj[u] if adj[u][v] > 0]
        if neighbors:
            v = neighbors[0]
            adj[u][v] -= 1
            adj[v][u] -= 1
            stack.append(v)
            yield "PUSH", (list(stack), list(path), (u, v))
        else:
```

```

        popped = stack.pop()
        path.append(popped)
        yield "POP", (list(stack), list(path), popped)

    final_path = path[::-1]
    yield "FINISHED", (final_path, path_type)

def find_eulerian_path_or_circuit_hierholzer(graph, is_directed=False):
    """
    Wrapper version of find_eulerian_path_or_circuit_hierholzer returning (path, path_type).
    """
    gen = find_eulerian_path_or_circuit_hierholzer_generator(graph, is_directed)
    last_val = ([], "")
    while True:
        try:
            step_type, data = next(gen)
            if step_type == "FINISHED":
                last_val = (data[0], data[1])
        except StopIteration as e:
            if e.value is not None:
                return e.value
            return last_val

```